

Assignment I - FIO

Student ID: 314551147

1 Question

Q0: Please run q0.fio on your own device and write down your output in report. Also specify the type of drive.

fio Output: Q0 Example (1G Direct Read)

```
read: IOPS=12.9k, BW=50.5MiB/s (52.9MB/s)(1024MiB/20283msec)
      lat (usec): min=41, max=15953, avg=76.91, stdev=80.67
```

Answer:

The drive used in this test is a **Solid State Drive (SSD)**.

According to the fio output, the 4K direct read achieves a bandwidth of **52.9 MB/s** with an extremely low average latency of just **76.91 μ s**.

What if it were an HDD?

If this test were run on a traditional HDD, the performance profile would be entirely different. Due to the mechanical overhead of moving the read/write head to scattered 4K sectors, the average latency would be measured in milliseconds. Consequently, the 4K read bandwidth would plummet, likely yielding only around 1 to 3 MB/s.

Q1: Is there any significant difference between read/randread? Why or why not? Please justify your answer in brief.

fio Output: Read vs. Random Read

```
read: IOPS=14.7k, BW=57.4MiB/s (60.2MB/s)(64.0MiB/1115msec)
      lat (usec): min=49, max=1148, avg=67.69, stdev=26.85
randread: IOPS=10.8k, BW=42.0MiB/s (44.0MB/s)(64.0MiB/1524msec)
      lat (usec): min=52, max=808, avg=92.44, stdev=28.77
```

Answer: Yes, there is a significant difference.

Sequential read bandwidth (**60.2 MB/s**) is noticeably higher than random read (**44.0 MB/s**).

Justification:

Even on SSDs, random reads are slower due to:

1. **Prefetch Failure:** Sequential reads allow the OS and SSD to predict and prefetch data. Random reads have unpredictable addresses, making prefetching ineffective.

2. **FTL Lookup Overhead:** Scattered LBA accesses force the SSD controller to frequently query the FTL mapping table, increasing latency.

What if it were an HDD?

The difference would be **drastic**. HDDs suffer from severe mechanical overhead during random accesses, often dropping random read speeds to below 1% of sequential speeds.

Q2: Is there any significant difference between write/random-write? Why or why not? Please justify your answer in brief.

fio Output: Write vs. Random Write

```
write: IOPS=17.0k, BW=66.4MiB/s (69.6MB/s)(64.0MiB/964msec)
      lat (usec): min=38, max=2142, avg=58.44, stdev=33.36
randwrite: IOPS=17.1k, BW=66.9MiB/s (70.2MB/s)(64.0MiB/956msec)
      lat (usec): min=33, max=1199, avg=57.81, stdev=27.30
```

Answer: No, there is no significant difference in this test.

Sequential write bandwidth (**69.6 MB/s**) and random write bandwidth (**70.2 MB/s**) are nearly identical, with very similar average latencies (**58.44 μ s** vs **57.81 μ s**).

Justification:

On modern SSDs, short random writes are just as fast as sequential writes due to:

1. **Log-Structured FTL:** The SSD's FTL does not overwrite data in place. It takes all incoming writes (whether sequential or random logically) and physically writes them **sequentially** to empty flash pages.
2. **SLC Caching / DRAM Buffers:** Incoming writes are quickly absorbed by the SSD's high-speed SLC cache. Since the test size is small (64MB), both workloads easily fit into the cache without triggering slow garbage collection or block erasures.

What if it were an HDD?

The difference would be **drastic**. HDDs must physically move the write head for random writes. The mechanical overhead would cause random write performance to crash compared to sequential writes.

Q3: Is there any significant difference between forward/backward write? Why or why not? Please justify your answer in brief.

fio Output: Forward vs. Backward Write

```
forward_write: IOPS=14.6k, BW=57.2MiB/s (60.0MB/s)(64.0MiB/1119msec)
      lat (usec): min=37, max=1035, avg=67.86, stdev=26.92
backward_write: IOPS=14.4k, BW=56.1MiB/s (58.9MB/s)(64.0MiB/1140msec)
      lat (usec): min=39, max=2235, avg=69.09, stdev=33.30
```

Answer: No, there is no significant difference.

Forward write bandwidth (**60.0 MB/s**) and backward write bandwidth (**58.9 MB/s**) are nearly identical, with very similar average latencies.

Justification:

On SSDs, logical writing direction does not affect physical writing speed due to:

1. **LBA Abstraction:** The FTL completely abstracts physical NAND pages. Writing to LBAs in reverse order means nothing to the physical hardware.
2. **Out-of-Place Writes:** Regardless of whether the logical blocks are written forward or backward, the SSD controller physically writes the incoming data sequentially to the next available erased pages.

What if it were an HDD?

The difference would be **drastic**. HDDs have platters that spin in only one direction. Backward writing forces the disk to wait for nearly a full mechanical rotation to reach the previous block, maximizing *Rotational Latency* and severely dropping throughput.

Q4: Buffered read vs. Nonbuffered read

fio Output: Direct I/O vs. Buffered I/O

```
[Sequential Read]
direct_io_seq: IOPS=15.0k, BW=58.5MiB/s (61.3MB/s)(64.0MiB/1094msec)
    lat (usec): min=46, max=1563, avg=66.42, stdev=31.35
buffered_io_seq: IOPS=334k, BW=1306MiB/s (1370MB/s)(64.0MiB/49msec)
    lat (nsec): min=430, max=643531, avg=2825.97, stdev=17628.87

[Random Read]
direct_io_rand: IOPS=11.3k, BW=44.1MiB/s (46.2MB/s)(64.0MiB/1452msec)
    lat (usec): min=50, max=2256, avg=87.95, stdev=43.24
buffered_io_rand: IOPS=10.8k, BW=42.4MiB/s (44.4MB/s)(64.0MiB/1510msec)
    lat (usec): min=50, max=1647, avg=91.60, stdev=30.48
```

Is there any significant difference between buffered/nonbuffered sequential read? Why or why not?

Answer: Yes, there is a massive difference.

Buffered sequential read (**1370 MB/s**) is drastically faster than non-buffered/Direct I/O (**61.3 MB/s**). Latency also drops from microseconds (μs) to nanoseconds (ns).

Justification: Buffered I/O utilizes the OS Page Cache and read-ahead (prefetching) mechanisms. The OS aggressively pre-fetches contiguous blocks of data from the disk into the RAM. Subsequent read requests hit the high-speed RAM cache directly instead of the slow physical disk. Non-buffered I/O completely bypasses this cache.

Replace sequential read with random read. Is there any significant difference between buffered/nonbuffered random read? Why or why not?

Answer: No, there is no significant difference.

Direct I/O (**46.2 MB/s**) and Buffered I/O (**44.4 MB/s**) perform almost identically in random reads.

Justification: Because random reads request data from unpredictable scattered addresses, the OS read-ahead mechanism becomes completely ineffective. Almost every read request results in a "cache miss" and must access the underlying physical SSD anyway. In this scenario, the OS Page Cache provides no performance benefit and merely adds a tiny amount of software overhead.

What if it were an HDD?

For **sequential reads**, the OS Page Cache is heavily relied upon to hide the HDD's slow mechanical speeds by aggressively pre-fetching data. For **random reads**, the buffer is nearly useless. The massive mechanical overhead dominates the process, making both buffered and non-buffered random reads extremely slow.

Q5: Is there any bandwidth trend between these jobs? Why or why not? Please justify your answer in brief.

fio Output: File Offsets (LBA)

job1 (offset=80%): IOPS=2585, BW=2586MiB/s (2711MB/s)
job2 (offset=60%): IOPS=3065, BW=3066MiB/s (3215MB/s)
job3 (offset=40%): IOPS=3190, BW=3190MiB/s (3345MB/s)
job4 (offset=20%): IOPS=2951, BW=2951MiB/s (3094MB/s)
job5 (offset=0%): IOPS=3200, BW=3200MiB/s (3355MB/s)

Answer: No, there is no significant bandwidth trend.

The bandwidth fluctuates slightly between **2711 MB/s** and **3355 MB/s**, showing roughly uniform performance across all file offsets without a clear directional degradation.

Justification:

Because the storage device is an SSD, logical file offsets do not correlate to physical disk geometry:

1. **LBA Abstraction:** Solid State Drives map Logical Block Addresses (LBAs) to physical NAND flash pages via the FTL. There are no physical "inner" or "outer" tracks.
2. **Uniform Access Time:** Unlike spinning disks, flash memory provides uniform access speeds. Accessing LBA 0 takes the same amount of physical time as accessing the highest LBA.

What if it were an HDD?

There would be a **clear downward trend**. HDDs use *Zone Bit Recording (ZBR)*. Outer tracks (offset=0%) have a larger circumference, packing more sectors per track, which results in much higher bandwidth. As the offset moves toward the inner tracks (offset=80%), the circumference shrinks, causing sequential read speeds to steadily decrease.

Q6: Blocksize (1K vs. 4K read)

fio Output: 1K vs. 4K Blocksize

```
[1K Blocksize]
read_1k: IOPS=16.2k, BW=15.8MiB/s (16.6MB/s)
        lat (usec): min=43, max=1197, avg=61.44, stdev=29.98

[4K Blocksize]
read_4k: IOPS=13.1k, BW=51.4MiB/s (53.9MB/s)
        lat (usec): min=41, max=1345, avg=75.46, stdev=54.00
```

Is there any significant difference between 4k/1k read? Why or why not?

Answer: Yes, there is a significant difference.

The 4K blocksize read (**53.9 MB/s**) is more than 3 times faster than the 1K blocksize read (**16.6 MB/s**).

Justification:

1. **Command Overhead:** Every I/O request carries a fixed software overhead. Reading 4KB of data with a 1K blocksize requires 4 separate requests, quadrupling the overhead compared to a single 4K request.
2. **Hardware Alignment:** Modern OS memory pages and SSD physical flash pages natively operate on 4KB units. A 1K blocksize forces unaligned, partial page accesses, which severely degrades storage throughput.

If you want to achieve the best performance in the condition above, how would you modify blocksize? Explain it briefly.

Answer: Increase the blocksize significantly (e.g., to 128KB, 1MB, or 4MB).

Explanation:

A larger blocksize dramatically reduces the instruction-to-payload ratio. Instead of overwhelming the CPU and SSD controller with millions of tiny I/O instructions, large blocks pack more data per request. This minimizes system overhead and allows the PCIe/SATA bandwidth to be fully saturated with actual data.

What if it were an HDD?

1. **1K vs. 4K:** The difference remains significant. Modern HDDs use "Advanced Format" which consists of 4KB physical sectors. Issuing 1KB reads forces the drive to read the full 4KB sector from the platter anyway, wasting mechanical effort.
2. **Best Performance:** To maximize HDD throughput, increasing the blocksize to 1MB or larger is absolutely crucial. HDDs are crippled by mechanical seek times. Huge block sizes ensure that once the read/write head finds the track, it can continuously stream massive amounts of sequential data without pausing to process new I/O commands.

Q7: Please explain how you achieve the fastest 1G nonbuffered read in brief.

fio Output: Fastest 1G Non-buffered Read

```
[4K Baseline]
```

```
4k_bs: IOPS=15.7k, BW=61.4MiB/s (64.3MB/s)(1024MiB/16688msec)
      lat (usec): min=41, max=2936, avg=63.36, stdev=28.14

[Optimized Fastest Read (1M)]
1m_bs: IOPS=6481, BW=6481MiB/s (6796MB/s)(1024MiB/158msec)
      lat (usec): min=750, max=10783, avg=4794.27, stdev=975.13
```

Answer:

By modifying the configuration, we achieved a peak bandwidth of **6796 MB/s**, finishing the 1GB read in just **0.158 seconds**. This is a massive exponential improvement over the baseline 4K read (**64.3 MB/s**, taking roughly 16.6 seconds).

Justification (How it was achieved):

To reach the hardware's physical limits, we combined three crucial parameters:

1. **Large Block Size (bs=1m)**: Maximizes the payload-to-overhead ratio. Instead of choking the system with millions of tiny 4K I/O commands, it floods the PCIe bus with large contiguous chunks of data.
2. **Asynchronous I/O (ioengine=libaio)**: Replaced the default synchronous engine. The CPU now fires off read requests continuously to the OS without blocking or waiting for each one to complete.
3. **High Queue Depth (iodepth=32)**: Kept 32 concurrent requests in the NVMe controller's queue at all times. This unlocks the SSD's **Internal Parallelism**, forcing multiple NAND flash dies and memory channels to fetch data simultaneously.

What if it were an HDD?

For a mechanical HDD, the large block size (**bs=1m**) would still help sequential performance immensely by minimizing *Seek Time*. However, the asynchronous engine and high queue depth (**iodepth=32**) would **not** trigger a massive speedup. HDDs lack internal parallelism—they only have one actuator arm reading from one physical location at a time. While a high queue depth allows the HDD to slightly optimize the read order using NCQ, the drive is still fundamentally limited by the mechanical spinning speed of the platter.

Bonus: Advanced Storage Comparison and Hardware Specification Analysis

Compare with multiple kinds of storage devices: Is there any significant difference between read/randread on storage devices? Why or why not?

Answer: The difference depends entirely on the storage medium:

- **On SSDs (like the PM9A1 tested)**: There is a difference, but it is purely electronic. Sequential reads are faster because they trigger the controller's *Read-Ahead* and *Multi-channel parallelism* more effectively. However, random reads are still very fast (> 40 MB/s in 4K) because there are no moving parts.

- **On HDDs:** The difference is **catastrophic**. Sequential read might reach 150-200 MB/s, but Random read drops to ≈ 1 MB/s.
- **Justification:** HDDs are limited by physics; random access requires the physical actuator arm to move and the platter to rotate for every single request. SSDs use FTL, making the "distance" between any two data blocks virtually zero.

Find the specs of your own hardware and determine if it is running to spec.

Hardware Identification: The tested drive is a **Samsung PM9A1 1TB (NVMe PCIe Gen 4.0 x4)**.

Official Specs vs. Test Results:

- **Official Sequential Read Speed:** Up to **7,000 MB/s**.
- **Our Test Result (Q7):** **6,796 MB/s**.

Analysis:

Yes, the hardware is running **perfectly to spec**. Our result of 6,796 MB/s is within 3% of the theoretical maximum.

Possible Theory for the minor gap:

1. **OS Overhead:** A small portion of the PCIe bandwidth is reserved for protocol headers and filesystem management within the Linux kernel.
2. **Thermal Throttling:** High-speed PCIe Gen 4 SSDs generate significant heat; if the drive reached a certain temperature during the 1G test, the controller might have slightly reduced speeds to protect the NAND components.
3. **CPU Bottleneck:** Even with `libaio` and `iodepth=32`, the overhead of context switching and interrupt handling on a single core can prevent reaching the absolute "hero number" listed on the box.